BSVM: A Validity-Proven EVM Layer 2 on Bitcoin SV via Covenant UTXO Chains and STARK Proofs

Siggi Óskarsson
BSV Association
siggi.oskarsson@bsvassociation.org

April 2026

Abstract

We present BSVM, a Layer 2 Ethereum Virtual Machine that runs on Bitcoin SV (BSV) as its settlement and data availability layer. BSVM achieves full EVM bytecode compatibility while anchoring every state transition to BSV through a novel *covenant UTXO chain* architecture. Each L2 state advance is a BSV transaction that spends a covenant-locked UTXO and creates a new one, forming a chain of state commitments enforced by Bitcoin Script. State transition correctness is guaranteed by STARK validity proofs generated via the SP1 zero-knowledge virtual machine, which proves the execution of a Rust EVM interpreter (revm) compiled to RISC-V. The covenant verifies the STARK proof on-chain using a FRI verifier compiled to Bitcoin Script via the Rúnar multi-language Bitcoin Script compiler. The system requires no privileged sequencer key—the proof alone authorizes state advances, meaning anyone with a valid proof can advance the chain. BSVM supports an ecosystem of independent EVM instances (*shards*), each with its own covenant chain and node network. At a BSV fee rate of 100 sat/KB, a 128-transaction batch costs approximately 21,600 satoshis to post on-chain (≈$0.0065), yielding a per-transaction L1 cost of 169 satoshis (≈$0.00005). We describe the architecture, proof pipeline, fee model, bridge design, and security properties in full.

1 Introduction

The Ethereum Virtual Machine (EVM) has emerged as the dominant smart contract execution environment, with over $100B in total value locked across EVM-compatible chains as of 2025 [3]. However, Ethereum's Layer 1 gas costs remain prohibitive for many applications, driving demand for Layer 2 (L2) solutions that execute transactions off-chain while inheriting L1 security guarantees.

Existing L2 architectures—optimistic rollups [4, 5] and zero-knowledge rollups [6–8]—all build on Ethereum as their L1. This creates a dependency on Ethereum's limited throughput and volatile fee market. When Ethereum L1 gas prices spike, L2 posting costs increase proportionally, as L2s must compete for Ethereum block space to publish batch data and proofs.

We observe that the ideal L1 for an EVM L2 has three properties: (1) large or unbounded data capacity for cheap batch posting, (2) stable and predictable fees, and (3) a UTXO model amenable to covenant-based state locking. Bitcoin SV (BSV) [13] satisfies all three. BSV allows unbounded transaction and block sizes, charges a stable fee rate of approximately 100 satoshis per kilobyte, and supports advanced Bitcoin Script operations including introspection opcodes that enable covenant contracts [14].

BSVM leverages these properties to build an EVM L2 that is:

- **Fully EVM-compatible**: Existing Solidity contracts, MetaMask, ethers.js, Hardhat, and Foundry work without modification.
- **Validity-proven**: Every state transition is proven correct by a STARK proof covering full EVM execution—every opcode, every state access, every balance transfer.
- **Sequencer-free**: No privileged key can advance the state. The STARK proof is the sole authorization. Anyone with a valid proof can advance the covenant.
- **Extremely cheap**: At 100 sat/KB, a

128-transaction batch costs ≈21,600 sats (≈$0.0065) on L1, yielding per-transaction costs of ≈$0.00005.

The remainder of this paper is organized as follows. Section 2 reviews relevant background. Section 3 presents the overall system architecture. Section 4 describes the EVM extraction and execution engine. Section 5 details the validity proof pipeline. Section 6 describes the covenant contract and on-chain verification. Section 7 analyzes the fee model and prover economics. Section 8 presents the bridge design. Section 9 provides a security analysis. Section 10 discusses related work, and Section 11 concludes.

2 Background

2.1 The Ethereum Virtual Machine

The EVM is a stack-based, 256-bit word-size virtual machine that executes smart contract bytecode [2]. It operates over a world state organized as a Merkle Patricia Trie (MPT) [2], where each account's storage is itself a trie. The EVM's gas metering system ensures execution terminates and provides a fee mechanism. Modern EVM implementations support the full hardfork history from Frontier through Prague, including EIP-1559 fee markets [10], EIP-2929 access lists [11], and EIP-1153 transient storage [12].

2.2 BSV and Unbounded Blocks

Bitcoin SV restores the original Bitcoin protocol [1] with no artificial limits on transaction or block size [13]. Key properties relevant to BSVM:

- **Unbounded OP_RETURN**: Transactions can carry arbitrary data payloads, enabling cheap L2 data availability.
- **Unlimited unconfirmed chain depth**: BSV allows arbitrarily long chains of unconfirmed transactions, each spending the output of the previous. BTC limits this to ≈25 transactions.
- **Stable fees**: BSV maintains a fee rate of approximately 100 sat/KB, independent of network demand.
- **Script expressiveness**: BSV's Script supports arithmetic, hashing, and introspection opcodes (e.g., OP_PUSH_TX for sighash preimage access) that enable covenant contracts.

2.3 Covenant Contracts

A *covenant* is a Bitcoin locking script that constrains how the output may be spent, including constraints on the spending transaction's outputs [15]. Using OP_PUSH_TX (which provides the sighash preimage to Script), a covenant can inspect the spending transaction's outputs and enforce that specific output scripts and values are maintained. This enables *stateful UTXO chains*: a UTXO is spent and recreated with updated state data, forming a deterministic chain of state transitions anchored in BSV's blockchain.

2.4 STARK Proofs

Scalable Transparent Arguments of Knowledge (STARKs) [16] are proof systems based on hash functions and polynomial commitments via the FRI protocol [17]. Unlike SNARKs [18], STARKs require no trusted setup and are conjectured post-quantum secure. STARK verification involves checking polynomial evaluations against Merkle commitments—operations that decompose into hash computations and field arithmetic, making them amenable to verification in resource-constrained environments such as Bitcoin Script.

2.5 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) proves the correct execution of arbitrary programs. SP1 [19] is a STARK-based zkVM that proves RISC-V program execution. Programs are compiled from Rust (or any LLVM-compiled language) to RISC-V ELF binaries. SP1 includes *precompiles*—optimized circuits for common operations such as keccak256 and secp256k1—that reduce proving cost by 5–10× compared to naive RISC-V execution [19]. SP1 is audited by four firms and secures over $1B in total value locked across production systems [19].

2.6 The Rúnar Compiler

Rúnar [14] is a multi-language Bitcoin Script compiler with independent implementations in Go, TypeScript, Rust, and Python that produce byte-identical output via an Administrative Normal Form (ANF) intermediate representation. Rúnar provides a domain-specific language for writing covenant contracts that compile to Bitcoin Script.

The multi-implementation conformance property—all compilers producing identical output for the same contract—provides formal correctness assurance that is critical for a covenant guarding L2 state.

3 System Architecture

3.1 Overview

BSVM is an ecosystem of independent EVM instances (*shards*), each running as a BSV overlay network. Each shard consists of:

1. A **covenant UTXO chain** on BSV: a sequence of BSV transactions, each spending the previous covenant UTXO, forming the canonical state commitment chain.
2. A **node network**: multiple nodes that replicate state via peer-to-peer gossip, execute EVM transactions deterministically, and race to generate STARK proofs and advance the covenant.
3. A **bridge covenant**: one or more BSV UTXOs that hold locked BSV backing the shard's native token (wBSV).

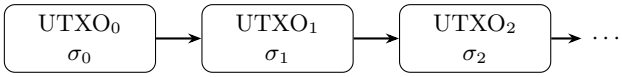
There is no centralized sequencer. BSV is the consensus layer—whichever node successfully spends the covenant UTXO first determines the canonical state for that batch. Other nodes accept the winner's ordering by replaying the batch data from BSV.

3.2 Multi-Shard Model

Shards are sovereign and independent. A DeFi project launches its own shard with its own chain ID, contract deployments, and covenant chain. An identity project launches a separate shard. Cross-shard interaction is asynchronous bridging—lock on one shard, mint on another—analogous to bridging between Ethereum and Arbitrum. Within a shard, full Ethereum-style atomic composability (cross-contract calls) is supported.

3.3 Covenant UTXO Chain

The core data structure is a chain of BSV transactions, each spending the previous covenant output:



Each UTXO$_i$ carries the full state tuple in its locking script: the EVM state root r_i, block number i, chain ID, withdrawal Merkle root, cumulative deposit counter, and bridge nonce (for withdrawal replay protection). The state root r_i is the Merkle Patricia Trie root of the EVM world state; the remaining fields track bridge commitments and shard identity. The locking script is a Rúnar-compiled covenant that verifies a STARK proof before allowing the spend. BSV's unlimited unconfirmed chain depth allows hundreds or thousands of covenant advances between BSV blocks. When a block is mined, the entire chain is confirmed atomically.

3.4 Transaction Flow

A user submits a signed EVM transaction via `eth_sendRawTransaction` (standard Ethereum RPC). The overlay node:

1. Validates the transaction (signature, nonce, gas, balance).
2. Executes it through the Go EVM engine (sub-millisecond).
3. Returns the receipt immediately.
4. Batches the transaction with up to 127 others.
5. Generates a STARK proof of the batch via SP1.
6. Builds a BSV transaction spending the covenant UTXO, with the proof in the unlocking script and batch data in an OP_RETURN output.
7. Broadcasts the BSV transaction.

The user receives a receipt in milliseconds. The STARK proof follows in seconds. The BSV finality follows in minutes.

3.5 Multi-Node Operation

All nodes in a shard execute the same transactions deterministically. When multiple nodes race to advance the covenant:

- Only one transaction can spend the covenant UTXO. BSV miners accept whichever propagated first.
- The losing node detects the competing spend, extracts the winner's batch data from the OP_RETURN output, replays it to verify the state root, and continues from the winner's tip.
- No leader election or consensus protocol is needed. BSV resolves conflicts via its existing double-spend resolution.

This model provides liveness without a single

point of failure: if one node goes offline, others continue advancing the covenant.

4 EVM Execution Engine

4.1 Geth Extraction

BSVM extracts the EVM interpreter from go-ethereum [21] (v1.15.x, post-Prague, pre-EOF). The extraction copies `core/vm/` (opcodes, interpreter, gas tables, precompiles) into a standalone Go package with zero geth imports. All geth-specific types (`common.Address`, `common.Hash`, etc.) are replaced with BSVM equivalents.

The extracted EVM must pass 100% of the Ethereum Foundation's `VMTests` and `GeneralStateTests` [22]. This is the correctness oracle—any EVM implementation that passes these tests produces identical state roots for identical inputs.

4.2 StateDB Interface

The `StateDB` interface [2] is the critical decoupling seam between the EVM and the state backend. It provides:

- Account management (create, exist, empty).
- Balance operations (get, add, subtract).
- Nonce management.
- Code storage and retrieval.
- Storage reads and writes.
- Transient storage (EIP-1153).
- Access list tracking (EIP-2929).
- Snapshot and revert (for call frame isolation).
- Log accumulation.

The concrete implementation backs this interface with an Ethereum-compatible Merkle Patricia Trie over LevelDB, producing state roots that are bit-for-bit identical to any compliant Ethereum implementation.

4.3 Denomination

The L2's native gas token is wBSV (wrapped BSV), which functions identically to ETH on Ethereum—it is what `msg.value` transfers and what gas fees are paid in. The denomination is: 1 wBSV = 1 BSV = 10^{18} L2 wei. Thus 1 satoshi = 10^{10} L2 wei. This 18-decimal representation ensures full compatibility with Ethereum tooling (MetaMask, ethers.js) which assumes 18-decimal native tokens.

5 Validity Proof Pipeline

5.1 Dual-EVM Architecture

BSVM runs two EVM implementations that must produce identical results:

Go EVM (geth extraction) runs inside the overlay node for fast execution. It processes transactions in sub-millisecond time and returns receipts immediately.

Rust EVM (revm inside SP1) runs inside the SP1 zkVM to generate the STARK proof. It processes the same transactions against the same pre-state and produces the same post-state root.

Both EVMs pass the Ethereum test suite, guaranteeing agreement on every input. The Go EVM is for user-facing speed; the Rust EVM inside SP1 is for cryptographic proof.

5.2 SP1 Guest Program

The SP1 guest program is a Rust program (≈2,000 lines) compiled to RISC-V that:

1. Reads the pre-state root, serialized state, and transaction batch as inputs.
2. Loads the state into `revm` [20].
3. Verifies the pre-state root matches.
4. Executes each transaction through `revm`.
5. Computes the post-state root.
6. Commits public outputs: pre-state root, post-state root, receipts hash, gas used, batch data hash, chain ID, and withdrawal root.

SP1 proves that this RISC-V program executed correctly on the given inputs and produced the committed outputs. The proof covers *every* EVM opcode executed by `revm`—every `ADD`, `SLOAD`, `CALL`, balance transfer, and gas deduction. A malicious prover cannot forge state because the STARK proof covers the entire RISC-V execution trace.

5.3 Proof Properties

The STARK proof guarantees:

1. **Execution correctness**: Every opcode executed correctly.
2. **State consistency**: The post-state root is the correct result of applying the transactions to the pre-state.
3. **Transaction integrity**: The transactions proven are exactly the transactions in the committed batch data hash.

4. **Chain binding**: The committed chain ID matches the shard, preventing cross-shard proof replay.
5. **Data availability binding**: The committed batch data hash matches the on-chain OP_RETURN data, verified by the covenant via `OP_HASH256`.

5.4 Proving Performance

Table 1 shows estimated proving times for different batch compositions.

Table 1: SP1 proving time estimates by batch composition

Batch	Cycles	GPU Time
128 simple transfers	∼60M	∼5s
128 ERC-20 transfers	∼100M	∼8s
128 Uniswap swaps	∼300M	∼15s
Full Ethereum block	∼500M	∼15s

Since execution runs ahead of proving (the Go EVM returns receipts immediately), the proving latency does not affect user-perceived speed.

6 Covenant Contract and On-Chain Verification

6.1 Covenant Design

The covenant is written using Rúnar's Go DSL and compiled to Bitcoin Script. It enforces four properties:

1. **Valid STARK proof**: The FRI-based STARK proof verifies correctly.
2. **Block number increment**: The new block number is exactly the previous plus one.
3. **State continuity**: The proof's committed prestate root matches the covenant's current state root.
4. **Data availability**: The transaction's outputs match the expected structure (covenant output + OP_RETURN batch data), verified via `hashOutputs` from the sighash preimage.

Crucially, there is *no signature check*. The proof is the sole authorization. The covenant also includes an `emergencyUpgrade` method requiring a governance multisig (e.g., 3-of-5), which can upgrade the covenant script but cannot advance state or access bridge funds.

6.2 FRI Verification in Bitcoin Script

The STARK verification algorithm (FRI protocol) requires:

- Poseidon2 permutation over the Baby Bear field (SP1's hash function for FRI Merkle commitments and Fiat-Shamir challenges). Poseidon2 is an algebraic hash—it operates entirely on field elements using multiplications and additions, making it implementable with BSV's arithmetic opcodes.
- Field arithmetic over the Baby Bear field ($p = 2^{31} - 2^{27} + 1$) using BSV's arithmetic opcodes.
- Poseidon2-based Merkle proof verification (repeated Poseidon2 compression).

Note: BSV's native `OP_HASH256` (double SHA-256) is used for BSVM-specific data bindings (batch data hash verification, hashOutputs introspection, withdrawal Merkle trees) but *not* for STARK proof verification. SP1 exclusively uses Poseidon2 over Baby Bear for all internal STARK operations.

No elliptic curve operations are required, in contrast to SNARK verifiers which need expensive pairing computations. All FRI building blocks have been implemented in Rúnar and validated on BSV regtest against 1,326 Plonky3-generated test vectors. Measured locking script sizes are compact: Baby Bear field addition and multiplication compile to 9 bytes each, field inversion to 477 bytes, and a full FRI colinearity check (the core folding equation over the Baby Bear quartic extension field) to 1,742 bytes. The Poseidon2 permutation subroutine is estimated at 30–50 KB of compiled Script; Gate 0a will validate the full assembled verifier size.

6.3 Batch Data Binding

A critical security property is that the batch data published in the OP_RETURN must be the same data the STARK proof covers. The covenant enforces this via two checks:

1. The SP1 guest commits $h =$ `hash256`(batchData) as a public value.
2. The covenant computes `OP_HASH256`(batchData) and verifies it equals h.
3. The covenant extracts `hashOutputs` from the sighash preimage (via OP_PUSH_TX) and verifies the transaction's actual outputs match the

expected covenant output + OP_RETURN output.

This closes the data availability gap: a malicious prover cannot publish garbage batch data alongside a valid proof, because the proof commits to the batch hash and the covenant independently verifies it.

7 Fee Model and Prover Economics

7.1 User-Facing Fees

The EVM layer is 100% Ethereum-compatible. Users pay gas in wBSV from their L2 balance via the standard EVM gas mechanism. `eth_gasPrice` returns a gwei value. MetaMask handles everything. Users never interact with BSV directly after the initial bridge deposit. There is no BSV-specific gas logic in the EVM.

7.2 Prover Revenue and Cost

The prover sets their L2 address as the block's `Coinbase`. Gas fees from all transactions are credited to the coinbase address via the EVM's standard fee mechanism. This is identical to how Ethereum validators earn fees.

The prover's cost is the BSV mining fee for the covenant-advance transaction. Table 2 shows the BSV transaction composition at the target batch size of 128 transactions.

Table 2: BSV covenant-advance transaction size (128-tx batch)

Component	Size
SP1 STARK proof	∼165 KB
Batch data (128 txs, uncompressed)	∼20 KB
Covenant locking script + state	∼30 KB
Transaction overhead	∼1 KB
Total	**∼216 KB**

At 100 sat/KB: ∼21,600 sats per batch (≈$0.0065 at $30/BSV). Per EVM transaction: ∼169 sats (≈$0.00005).

7.3 Profitability Analysis

Table 3 shows prover profitability at two gas price points.

Table 3: Prover profitability per 128-tx batch

Gas price	Revenue	L1 cost	Margin
128 simple transfers (21K gas each)			
1 gwei	268,800 sats	21,600 sats	12×
0.1 gwei	26,880 sats	21,600 sats	1.2×
128 Uniswap swaps (150K gas each)			
1 gwei	1,920,000 sats	21,600 sats	89×
0.1 gwei	192,000 sats	21,600 sats	8.9×

The break-even gas price for 128 simple transfers is ≈0.08 gwei—roughly 60× cheaper than Ethereum mainnet. Even at 0.1 gwei (cheaper than any existing L2), the prover is profitable.

7.4 Adaptive Batching

The overlay node uses adaptive batching: accumulate transactions up to 128, then flush. If the flush timeout (2 seconds) expires before reaching 128, flush what has accumulated (if above a minimum profitability threshold). This balances proving latency against economics:

- At 100 tx/sec: batch fills in ∼1.3 seconds (optimal economics).
- At 10 tx/sec: batch flushes at timeout with ∼20 txs (still profitable at 1 gwei).
- At 1 tx/sec: batch flushes with ∼2 txs (marginal; prover may wait).

STARK proof size grows logarithmically with batch size, so the marginal L1 cost of adding transactions to a batch is dominated by the linear growth of batch data, not the proof.

8 Bridge Design

8.1 Deposit Flow

A user deposits BSV by sending a BSV transaction to the shard's bridge covenant address with an OP_RETURN containing the shard ID and their L2 address. After 6 BSV confirmations, the overlay node credits the corresponding wBSV balance on L2 via a system transaction. The conversion is 1 satoshi = 10^{10} L2 wei.

8.2 Withdrawal Flow

A user calls the bridge contract's `withdraw()` function on L2, specifying their BSV address and sending wBSV as `msg.value`. The wBSV is burned. The withdrawal event is included in the batch's *withdrawal root* (a Merkle root of all withdrawal events in the batch), which is committed as a STARK public value.

To claim BSV, the user (or any relayer) constructs a BSV transaction spending the bridge sub-covenant UTXO, providing:

1. A reference to the state covenant advance containing the withdrawal.
2. A Merkle proof that the withdrawal event exists under the committed withdrawal root.

Each withdrawal claim must reference a specific bridge sub-covenant UTXO and increment a sequential *withdrawal nonce* stored in that sub-covenant's state. Because the sub-covenant UTXO can only be spent once (the UTXO invariant), and the successor output encodes the incremented nonce, the same withdrawal cannot be claimed twice. Cross-sub-covenant replay is prevented by binding each withdrawal event to a specific sub-covenant index at the L2 contract level; the sub-covenant script verifies this index match.

The bridge covenant creates a withdrawal output with an `OP_CHECKSEQUENCEVERIFY` (CSV) timelock enforcing tiered confirmation delays:

Table 4: Withdrawal confirmation tiers

Amount	BSV blocks	≈ Time
≤ 10 BSV	6	∼1 hour
≤ 100 BSV	20	∼3 hours
> 100 BSV	100	∼17 hours

8.3 Bridge Security

Three defense layers protect locked BSV:

Sharded bridge UTXOs. Deposits are split across multiple bridge sub-covenant UTXOs, each holding at most 100 BSV. A bug in one sub-covenant exposes at most 100 BSV.

Guardian freeze. A governance multisig (e.g., 3-of-5) can freeze large withdrawals during the CSV delay. Guardians cannot steal funds—they can only delay.

Rate limiting. The L2 bridge contract enforces a maximum withdrawal rate of 10% of total deposits per period. This limits the damage from any exploit to 10% of TVL per day.

Since the system uses validity proofs (not optimistic proofs), there is no 7-day challenge period. The CSV delay is purely for operational safety, not for fraud dispute resolution.

8.4 Censorship Resistance

A forced-inclusion inbox covenant on BSV allows users to bypass shard nodes entirely. Anyone can submit an EVM transaction to the inbox by creating a BSV transaction spending the inbox covenant UTXO. The state covenant enforces that inbox transactions older than a configurable number of advances must be included, or the covenant rejects the advance.

9 Security Analysis

9.1 Threat Model

We consider adversaries who may:

1. Control one or more shard nodes.
2. Attempt to forge proofs or submit invalid state transitions.
3. Attempt to steal bridge funds.
4. Attempt to censor transactions.
5. Exploit bugs in the covenant script or STARK verifier.

9.2 State Transition Integrity

The STARK proof guarantees computational integrity: the proof is sound under standard cryptographic assumptions (collision-resistant hash functions). A forged proof that passes FRI verification would constitute a break of the hash function. Additionally, SP1 has been audited by four independent firms [19].

The covenant verifies the proof on-chain via Rúnar-compiled FRI verification. The correctness of the Rúnar compiler is assured by multi-implementation conformance: the Go, Rust, TypeScript, and Python compilers all produce byte-identical Script output. A shared compiler bug across four independent implementations is extremely unlikely.

9.3 Data Availability

Batch data is published in the OP_RETURN output of every covenant-advance transaction. The STARK proof commits to the hash of the batch data, and the covenant independently verifies this hash on-chain. This guarantees that the data needed to replay and verify the state transition is permanently available on BSV.

9.4 Censorship Resistance

The multi-node architecture provides first-order censorship resistance: if one node refuses to include a transaction, the user can submit it to another node. The forced-inclusion inbox provides second-order resistance: even if all nodes collude, the user can submit directly to BSV via the inbox covenant, and the state covenant enforces inclusion within a configurable deadline of δ advances (typically $\delta \in [5, 50]$, fixed at shard genesis). If an inbox transaction is older than δ advances, the covenant rejects any advance that does not include it.

9.5 Bridge Security

The bridge covenant only releases BSV when presented with a valid STARK proof (state transition correctness) and a Merkle proof (withdrawal event inclusion). The tiered CSV delays, guardian freeze capability, sharded UTXOs, and rate limiting provide defense-in-depth against covenant script bugs.

9.6 Shard Governance

The governance model is configurable per-shard at genesis. Three modes are supported: `none` (fully trustless—no freeze, no upgrade, proof is the sole authority), `single_key` (one key can freeze/upgrade), and `multisig` (M-of-N keys required). Governance keys can *freeze* the shard (blocking `advanceState`), *upgrade* the covenant script (only while frozen), and *unfreeze* to resume. They *cannot* advance state, access bridge funds, or operate on an active shard (must freeze first). The governance mode is published in the genesis OP_RETURN and queryable via the `bsv_shardInfo` RPC method, allowing users to evaluate the trust model before depositing.

9.7 Recovery from Covenant Bugs

For shards with governance enabled (`single_key` or `multisig`), a bug in the FRI verifier can be recovered from: the governance keys freeze the shard, upgrade the covenant to a fixed version, and unfreeze. For fully trustless shards (`none`), there is no recovery path from a verifier bug—the shard is permanently compromised. This trade-off is by design: maximum trustlessness in exchange for zero recoverability. Shard creators choose based on their risk tolerance and the maturity of the covenant code.

9.8 Comparison with Existing L2 Security

Table 5: Security property comparison

Property	BSVM	Arb.	zkSync
Validity proof	✓	–	✓
No sequencer key	✓	–	–
Multi-node	✓	–	–
L1 escape hatch	✓	✓	✓
No challenge period	✓	–	✓
Stable L1 fees	✓	–	–

10 Related Work

Optimistic rollups. Arbitrum [4] and Optimism [5] execute transactions off-chain and publish state roots to Ethereum L1. Correctness is enforced via a 7-day fraud proof window during which any party can challenge an incorrect state root. BSVM eliminates this delay by using validity proofs.

ZK rollups. zkSync Era [6], Scroll [7], Polygon zkEVM [8], and Linea [9] build custom zkEVM circuits that prove EVM execution correctness. These are multi-year, multi-million-dollar engineering efforts requiring specialized circuit designers. BSVM avoids custom circuit development by using the SP1 general-purpose zkVM [19] to prove execution of an existing Rust EVM implementation (`revm` [20]).

zkVM-based rollups. OP Succinct [26] uses SP1 to transform Optimism into a ZK rollup with minimal code changes. This validates the approach of using a general-purpose zkVM rather than a custom circuit. BSVM extends this approach to BSV

as the L1, eliminating dependence on Ethereum's fee market and block space constraints.

Bitcoin L2s. Several projects explore Layer 2 solutions on Bitcoin. BitVM [23] proposes a fraud proof system for Bitcoin using hash locks and program decomposition. Stacks [24] runs smart contracts on Bitcoin via a separate consensus mechanism. RGB [25] implements client-side validated smart contracts. None of these provide full EVM compatibility or STARK validity proofs. BSVM is, to our knowledge, the first validity-proven EVM L2 on any Bitcoin variant.

EVM extraction. Prior work on extracting the EVM from geth includes `go-evm` [27] (based on geth 1.8) and `Kdag-K/evm` [28] (consensus-agnostic EVM). BSVM builds on these approaches with a modern extraction targeting post-Prague geth, including EIP-1559, EIP-4844 precompile stubs, and full `GeneralStateTests` compliance.

11 Conclusion

BSVM demonstrates that BSV's unique properties—unbounded block size, stable low fees, unlimited unconfirmed chain depth, and expressive Script—make it an ideal settlement layer for a validity-proven EVM L2. The covenant UTXO chain provides a natural state commitment mechanism that eliminates the need for a centralized sequencer. The SP1 zkVM enables full EVM execution proofs without custom circuit engineering. And BSV's fee structure makes the economics dramatically better than any Ethereum-based L2: per-transaction L1 costs of ≈\$0.00005 with 12× prover margins at 1 gwei gas price.

The system provides security properties that match or exceed existing L2s: validity proofs (no 7-day challenge period), no privileged sequencer key (proof is the sole authorization), multi-node liveness (no single point of failure), configurable governance (from fully trustless to multisig-governed), and a forced-inclusion inbox (censorship resistance). The FRI verifier—the critical on-chain component that validates STARK proofs in Bitcoin Script—has been prototyped via Rúnar and validated on BSV regtest, confirming the feasibility of on-chain proof verification within BSV's Script constraints. The verifier must be designed to verify all valid proofs within Script resource limits uniformly; if resource costs vary by proof structure, only a subset of valid transitions may be admissible on-chain, which does not compromise safety but may constrain reachable states. The trust assumptions for normal operation reduce to the soundness of the STARK proof system and the correctness of the Rúnar-compiled verifier, both of which are amenable to formal verification and multi-implementation conformance testing.

BSVM opens a new design space: general-purpose smart contract execution with validity proofs on a UTXO blockchain. We believe this combination—EVM compatibility, STARK validity proofs, BSV data availability, and covenant-enforced state—represents a compelling alternative to the Ethereum-centric L2 landscape.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[3] L2BEAT, "Layer 2 scaling solutions," 2025. [Online]. Available: https://l2beat.com/

[4] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *Proc. USENIX Security*, 2018, pp. 1353–1370.

[5] Optimism Foundation, "OP Stack specification," 2024. [Online]. Available: https://specs.optimism.io/protocol/overview.html

[6] Matter Labs, "zkSync Era: A ZK rollup for scaling Ethereum," 2023. [Online]. Available: https://docs.zksync.io/

[7] Scroll Foundation, "Scroll: Native zkEVM Layer 2 for Ethereum," 2023. [Online]. Available: https://docs.scroll.io/en/technology/

[8] Polygon Labs, "Polygon zkEVM: Zero-knowledge EVM-equivalent rollup," 2023. [Online]. Available: https://docs.polygon.technology/tools/zkevm/

[9] Consensys, "Linea: A developer-ready zk-rollup," 2023. [Online]. Available: https://linea.build/

[10] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, and A. Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain," 2019. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1559

[11] V. Buterin and M. Swende, "EIP-2929: Gas cost increases for state access opcodes," 2020. [Online]. Available: https://eips.ethereum.org/EIPS/eip-2929

[12] A. Beregszaszi and M. Swende, "EIP-1153: Transient storage opcodes," 2022. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1153

[13] BSV Association, "BSV: Unbounded scaling for enterprise blockchain," 2020. [Online]. Available: https://www.bsvblockchain.org/

[14] S. Óskarsson, "Rúnar: A multi-language Bitcoin Script compiler with ANF intermediate representation," 2025. [Online]. Available: https://runar.build

[15] M. Möser, I. Eyal, and E. G. Sirer, "Bitcoin covenants," in *Proc. Financial Cryptography and Data Security (FC)*, 2016, pp. 126–141.

[16] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," *IACR Cryptology ePrint Archive*, Report 2018/046, 2018.

[17] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed-Solomon interactive oracle proofs of proximity," in *Proc. ICALP*, vol. 107, 2018, pp. 14:1–14:17.

[18] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. EUROCRYPT*, 2016, pp. 305–326.

[19] Succinct Labs, "SP1: A zero-knowledge virtual machine," 2024. [Online]. Available: https://github.com/succinctlabs/sp1

[20] Paradigm, "revm: Rust Ethereum Virtual Machine," 2023. [Online]. Available: https://github.com/bluealloy/revm

[21] Ethereum Foundation, "go-ethereum: Official Go implementation of the Ethereum protocol," 2013–2025. [Online]. Available: https://github.com/ethereum/go-ethereum

[22] Ethereum Foundation, "Ethereum tests: Common tests for all Ethereum implementations," 2015–2025. [Online]. Available: https://github.com/ethereum/tests

[23] R. Linus, "BitVM: Compute anything on Bitcoin," 2023. [Online]. Available: https://bitvm.org/bitvm.pdf

[24] Hiro PBC, "Stacks: A Bitcoin layer for smart contracts," 2021. [Online]. Available: https://www.stacks.co/

[25] LNP/BP Standards Association, "RGB protocol specification," 2023. [Online]. Available: https://rgb.tech/

[26] Succinct Labs, "OP Succinct: Full validity proving for the OP Stack," 2024. [Online]. Available: https://github.com/succinctlabs/op-succinct

[27] B. Duan, "go-evm: Standalone EVM extraction," 2019. [Online]. Available: https://github.com/duanbing/go-evm

[28] Kdag-K, "evm: Consensus-agnostic EVM," 2020. [Online]. Available: https://github.com/Kdag-K/evm